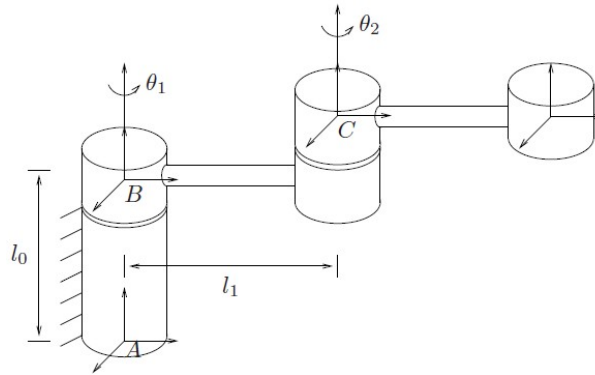


MLS_ex_2_6

April 13, 2026

1 MLS ex 2.6 p. 60

(Developed by NJ Theron, April 2026.)



In its solution to this example problem, MLS states that

$$V_{ab}^s = \begin{bmatrix} v_{ab} \\ \omega_{ab} \end{bmatrix} \dot{\theta}_1$$

and

$$V_{bc}^s = \begin{bmatrix} v_{bc} \\ \omega_{bc} \end{bmatrix} \dot{\theta}_2 ,$$

and they then give expressions for the column vectors v_{ab} , ω_{ab} , v_{bc} and ω_{bc} . All of these are certainly not incorrect, but do not follow directly from anything that was derived earlier in the text (as far as I can see). This script serves to clarify how MLS derives the above and the rest of their solution, which seems quite a bit more complicated than what appears at first glance.

```
[2]: from sympy import symbols, simplify, diff, Function
      from sympy.matrices import Matrix, BlockMatrix, zeros, eye
      from robotics_functions_sym import (hat, Rodrigues, wedge, vee, gmat,
      gmati, Adgg)
```

```
[3]: t, l_0, l_1 = symbols('t, l_0, l_1', real=True)
      theta1 = Function('theta1')(t)
      theta1
```

```
[3]: theta1(t)
```

```
[4]: theta2 = Function('theta2')(t)
      theta2
```

```
[4]:  $\theta_2(t)$ 
```

The next cell calculates the general rotation matrices corresponding to the rotations of frames B and C through the angles θ_1 and θ_2 , respectively, both rotations about the unit vector $u = [0 \ 0 \ 1]^T$.

```
[6]: u = Matrix([[0],[0],[1]])
      exphutheta1 = Rodrigues(u,theta1) # creating rotation matrix with Rodrigues'
      # formula, MLS eq. (2.14) p. 28; u is the direction unit vector
      exphutheta2 = Rodrigues(u,theta2)
      exphutheta2
```

```
[6]: 
$$\begin{bmatrix} \cos(\theta_2(t)) & -\sin(\theta_2(t)) & 0 \\ \sin(\theta_2(t)) & \cos(\theta_2(t)) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

```

The next two code cells calculate the initial configurations of frames B and C , $g_{ab}(0)$ and $g_{bc}(0)$, respectively, with the function `gmat`. The function `gmat` is not a standard Python function, but one developed by myself, in my file `robotics_functions_sym.py`. I do not make this available to students, as students need to develop their own utility functions like this. The function `gmat` packs its input variables into a homogeneous representation of the rigid body displacement transformation as in MLS equation (2.23) p. 36 and returns the resulting g matrix to the calling program.

In the initial configurations, $\theta_1 = \theta_2 = 0$ and hence the rotation matrices are both identity matrices.

```
[8]: p_ab0 = Matrix([[0],[0],[1_0]])
      gab0 = gmat(eye(3),p_ab0)
      gab0
```

```
[8]: 
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & l_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```

```
[9]: p_bc0 = Matrix([[0],[1_1],[0]]) # I assume here that the frame $C$ origin has
      # the same $z$ coordinate as frame $B$.
      gbc0 = gmat(eye(3),p_bc0)
      gbc0
```

```
[9]: 
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & l_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```

From eq. (2.45) p. 49:

$$g_{ab}(\theta_1) = e^{\hat{\xi}_1 \theta_1} g_{ab}(0)$$

$$g_{bc}(\theta_2) = e^{\hat{\xi}_2 \theta_2} g_{bc}(0)$$

To calculate $e^{\hat{\xi}_1 \theta_1}$ and $e^{\hat{\xi}_2 \theta_2}$ we shall use MLS eq. (2.36) p. 42. On p. 47 MLS states (and this has been proven in the slides set for March 16, 2026) that if we choose $v = -\hat{\omega}q + h\omega$, the twist $\xi = (v, \omega)$ defined in eq. (2.36) generates the screw defined in eq. (2.40). For a revolute joint, like the two joints in this example, $h = 0$ and it was shown in the notes set “on_MLS_Chapter_2_Section_2_3.pdf” (on Moodle) that, when $h = 0$, the term $\omega\omega^T v\theta = 0$. Therefore, for this example (and for a revolute joint, in general)

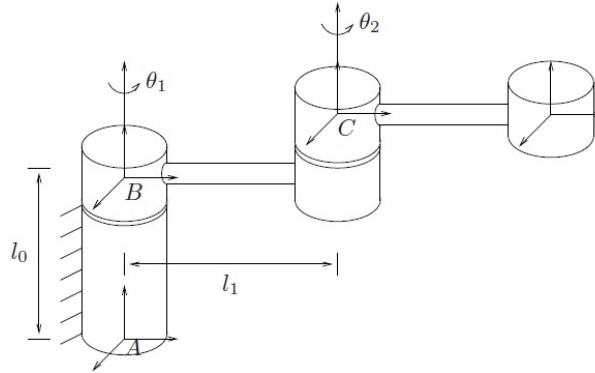
$$v_1 = -\hat{\omega}_1 q_1$$

and

$$e^{\hat{\xi}_1 \theta_1} = \begin{bmatrix} e^{\hat{\omega}_1 \theta_1} & (I - e^{\hat{\omega}_1 \theta_1})\hat{\omega}_1 v_1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} e^{\hat{\omega}_1 \theta_1} & (I - e^{\hat{\omega}_1 \theta_1})q_1 \\ 0 & 1 \end{bmatrix},$$

by equations (2.36) and (2.40). Similarly for $e^{\hat{\xi}_2 \theta_2}$. Note that in this case, $\omega_1 = \omega_2$ and are equal to the unit vector u in the code (and these are not angular velocities and should therefore not be confused with angular velocities).

q_1 is a point (any point) on the axis of ξ_1 (expressed in frame A) and q_2 is a point (any point) on the axis of ξ_2 (expressed in frame B).



```
[11]: q1 = Matrix([[0],[0],[0]])
      q2 = Matrix([[0],[1_1],[0]])
```

```
[12]: v1 = -hat(u)*q1
      v1
```

```
[12]: [0]
      [0]
      [0]
```

```
[13]: v2 = -hat(u)*q2
      v2
```

```
[13]: [l_1]
      [0]
      [0]
```

$\xi_1 :$

```
[15]: xi1 = Matrix(BlockMatrix([[v1],[u]]))
      xi1
```

[15]:
$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

The next code cell calculates $e^{\hat{\xi}_1 \theta_1}$ using $\omega_1 = u$ and

$$e^{\hat{\xi}_1 \theta_1} = \begin{bmatrix} e^{\hat{\omega}_1 \theta_1} & (I - e^{\hat{\omega}_1 \theta_1})q_1 \\ 0 & 1 \end{bmatrix}.$$

```
[17]: twistexp1 = Matrix(BlockMatrix(
    [[exphutheta1, (eye(3) - exphutheta1)*q1], [zeros(1,3), Matrix([1])]]
    ))
twistexp1
```

[17]:
$$\begin{bmatrix} \cos(\theta_1(t)) & -\sin(\theta_1(t)) & 0 & 0 \\ \sin(\theta_1(t)) & \cos(\theta_1(t)) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

ξ_2 :

```
[19]: xi2 = Matrix(BlockMatrix([[v2], [u]]))
xi2
```

[19]:
$$\begin{bmatrix} l_1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

The next code cell calculates $e^{\hat{\xi}_2 \theta_2}$ using $\omega_2 = u$ and

$$e^{\hat{\xi}_2 \theta_2} = \begin{bmatrix} e^{\hat{\omega}_2 \theta_2} & (I - e^{\hat{\omega}_2 \theta_2})q_2 \\ 0 & 1 \end{bmatrix}.$$

```
[21]: twistexp2 = Matrix(BlockMatrix(
    [[exphutheta2, (eye(3) - exphutheta2)*q2], [zeros(1,3), Matrix([1])]]
    ))
twistexp2
```

[21]:
$$\begin{bmatrix} \cos(\theta_2(t)) & -\sin(\theta_2(t)) & 0 & l_1 \sin(\theta_2(t)) \\ \sin(\theta_2(t)) & \cos(\theta_2(t)) & 0 & l_1 \cdot (1 - \cos(\theta_2(t))) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

As stated above,

$$g_{ab}(\theta_1) = e^{\hat{\xi}_1 \theta_1} g_{ab}(0)$$

$$g_{bc}(\theta_2) = e^{\hat{\xi}_2 \theta_2} g_{bc}(0)$$

```
[23]: gab = twistexp1*gab0
      gab
```

```
[23]:
```

$$\begin{bmatrix} \cos(\theta_1(t)) & -\sin(\theta_1(t)) & 0 & 0 \\ \sin(\theta_1(t)) & \cos(\theta_1(t)) & 0 & 0 \\ 0 & 0 & 1 & l_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
[24]: gbc = twistexp2*gbc0
      gbc
```

```
[24]:
```

$$\begin{bmatrix} \cos(\theta_2(t)) & -\sin(\theta_2(t)) & 0 & 0 \\ \sin(\theta_2(t)) & \cos(\theta_2(t)) & 0 & l_1 \cdot (1 - \cos(\theta_2(t))) + l_1 \cos(\theta_2(t)) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1.0.1 Calculation of V_{ab}^s

MLS p. 54: $\hat{V}_{ab}^s = \dot{g}_{ab} g_{ab}^{-1}$:

```
[26]: Whatabs = simplify(diff(gab,t)*gmati(gab))
      Whatabs
```

```
[26]:
```

$$\begin{bmatrix} 0 & -\frac{d}{dt}\theta_1(t) & 0 & 0 \\ \frac{d}{dt}\theta_1(t) & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

In the code cell above I used my function `gmati` (in my file `robotics_functions_sym.py`) to calculate g_{ab}^{-1} using MLS's development of the inverse of g on p. 37.

The statement of MLS in the 3rd line of their solution to this problem (p. 60), "Since this motion is a screw motion...", seems to indicate that they had in mind the following, alternative way of calculating $\hat{V}_{ab}^s$, namely using $\hat{V}_{ab}^s = \hat{\xi}_1 \dot{\theta}_1$ (MLS p. 58):

```
[28]: hat_xi1 = wedge(xi1)
      Whatabs_2 = hat_xi1*diff(theta1,t) # Really the calculation suggested in
                                          # example 2.6

      shouldbezero4times4 = Whatabs - Whatabs_2
      shouldbezero4times4
```

```
[28]:
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

```
[29]: Vabs = vee(Vhatabs)  # vee operator function, like in MLS eq. (2.30)
      # p. 41
Vabs
```

```
[29]: [ 0
      0
      0
      0
      0
       $\frac{d}{dt}\theta_1(t)$ ]
```

The last result above is the same as V_{ab}^s in example 2.6 p. 60.

1.0.2 Calculation of V_{bc}^s

MLS p. 54: $\hat{V}_{bc}^s = \dot{g}_{bc}g_{bc}^{-1}$:

```
[31]: Vhatbcs = simplify(diff(gbc,t)*gmati(gbc))
Vhatbcs
```

```
[31]: [ 0       $-\frac{d}{dt}\theta_2(t)$   0   $l_1\frac{d}{dt}\theta_2(t)$ 
       $\frac{d}{dt}\theta_2(t)$       0      0      0
      0      0      0      0
      0      0      0      0]
```

Once again, the statement of MLS in the 3rd line of their solution to this problem (p. 60), “Since this motion is a screw motion...”, seems to indicate that they had in mind the following, alternative way of calculating $\hat{V}_{bc}^s$, namely using $\hat{V}_{bc}^s = \hat{\xi}_2\dot{\theta}_2$ (MLS p. 58):

```
[33]: hat_xi2 = wedge(xi2)
Vhatbcs_2 = hat_xi2*diff(theta2,t)  # Really the calculation suggested in
                                     # example 2.6
shouldbezero4times4 = Vhatbcs - Vhatbcs_2
shouldbezero4times4
```

```
[33]: [0 0 0 0]
      [0 0 0 0]
      [0 0 0 0]
      [0 0 0 0]
```

```
[34]: Vbcs = vee(Vhatbcs)  # vee operator function, like in MLS eq. (2.30)
      # p. 41
Vbcs
```

```
[34]: [ $l_1\frac{d}{dt}\theta_2(t)$ 
      0
      0
      0
      0
       $\frac{d}{dt}\theta_2(t)$ ]
```

The last result above is the same as V_{bc}^s in example 2.6 p. 60.

Note that the “spatial” velocity V_{bc}^s is “spatial” only in the sense that it is expressed in frame B , as if frame B is the spatial frame. But frame B is not an inertial frame, as it moves (and possibly rotates and accelerates) relative to the (assumed) inertial frame, frame A .

In the next code cell, I am calculating $\hat{V}_{ac}^s$ using the unnumbered equation close to the top of p. 59:

$$\hat{V}_{ac}^s = \hat{V}_{ab}^s + g_{ab} \hat{V}_{bc}^s g_{ab}^{-1}.$$

```
[36]: gabi = gmati(gab) # calculate the inverse of gab, MLS p. 37.
```

```
Vhatacs = simplify(Vhatabs + gab*Vhatbcs*gabi)
Vhatacs
```

```
[36]:
```

$$\begin{bmatrix} 0 & -\frac{d}{dt}\theta_1(t) - \frac{d}{dt}\theta_2(t) & 0 & l_1 \cos(\theta_1(t)) \frac{d}{dt}\theta_2(t) \\ \frac{d}{dt}\theta_1(t) + \frac{d}{dt}\theta_2(t) & 0 & 0 & l_1 \sin(\theta_1(t)) \frac{d}{dt}\theta_2(t) \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

```
[37]: Vacs = vee(Vhatacs) # vee operator function, like in MLS eq. (2.30)
# p. 41
Vacs
```

```
[37]:
```

$$\begin{bmatrix} l_1 \cos(\theta_1(t)) \frac{d}{dt}\theta_2(t) \\ l_1 \sin(\theta_1(t)) \frac{d}{dt}\theta_2(t) \\ 0 \\ 0 \\ 0 \\ \frac{d}{dt}\theta_1(t) + \frac{d}{dt}\theta_2(t) \end{bmatrix}$$

The last result above is the same as V_{ac}^s in example 2.6 p. 60.

1.1 Alternative calculation

(as on MLS p. 60)

In this I use my function `Adgg` (in my file `robotics_functions_sym.py`) to calculate the adjoint transformation matrix of g_{ab} , using the definition eq. (2.58) (p. 55).

```
[39]: Ad_gab = Adgg(gab)
Ad_gab
```

```
[39]:
```

$$\begin{bmatrix} \cos(\theta_1(t)) & -\sin(\theta_1(t)) & 0 & -l_0 \sin(\theta_1(t)) & -l_0 \cos(\theta_1(t)) & 0 \\ \sin(\theta_1(t)) & \cos(\theta_1(t)) & 0 & l_0 \cos(\theta_1(t)) & -l_0 \sin(\theta_1(t)) & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & \cos(\theta_1(t)) & -\sin(\theta_1(t)) & 0 \\ 0 & 0 & 0 & \sin(\theta_1(t)) & \cos(\theta_1(t)) & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

```
[40]: Vacs_2 = Vabs + Ad_gab*Vbcs
Vacs_2
```

```
[40]:
```

$$\begin{bmatrix} l_1 \cos(\theta_1(t)) \frac{d}{dt} \theta_2(t) \\ l_1 \sin(\theta_1(t)) \frac{d}{dt} \theta_2(t) \\ 0 \\ 0 \\ 0 \\ \frac{d}{dt} \theta_1(t) + \frac{d}{dt} \theta_2(t) \end{bmatrix}$$

```
[41]: shouldbezero6times1 = Vacs_2 - Vacs
      shouldbezero6times1
```

```
[41]:  $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ 
```

1.2 Yet another alternative calculation

MLS p. 54: $\hat{V}_{ac}^s = \dot{g}_{ac} g_{ac}^{-1}$

By using eq. (2.24) p. 37, we can calculate g_{ac} :

$$g_{ac} = g_{ab} g_{bc} \cdot$$

```
[43]: gac = gab*gbc
      gac[:, :2]
```

```
[43]:  $\begin{bmatrix} -\sin(\theta_1(t)) \sin(\theta_2(t)) + \cos(\theta_1(t)) \cos(\theta_2(t)) & -\sin(\theta_1(t)) \cos(\theta_2(t)) - \sin(\theta_2(t)) \cos(\theta_1(t)) \\ \sin(\theta_1(t)) \cos(\theta_2(t)) + \sin(\theta_2(t)) \cos(\theta_1(t)) & -\sin(\theta_1(t)) \sin(\theta_2(t)) + \cos(\theta_1(t)) \cos(\theta_2(t)) \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$ 
```

```
[44]: gac[:, 2:]
```

```
[44]:  $\begin{bmatrix} 0 & -(l_1 \cdot (1 - \cos(\theta_2(t))) + l_1 \cos(\theta_2(t))) \sin(\theta_1(t)) \\ 0 & (l_1 \cdot (1 - \cos(\theta_2(t))) + l_1 \cos(\theta_2(t))) \cos(\theta_1(t)) \\ 1 & l_0 \\ 0 & 1 \end{bmatrix}$ 
```

```
[45]: Vhatacs_3 = simplify(diff(gac,t)*gmati(gac))
      Vhatacs_3
```

```
[45]:  $\begin{bmatrix} 0 & -\frac{d}{dt} \theta_1(t) - \frac{d}{dt} \theta_2(t) & 0 & l_1 \cos(\theta_1(t)) \frac{d}{dt} \theta_2(t) \\ \frac{d}{dt} \theta_1(t) + \frac{d}{dt} \theta_2(t) & 0 & 0 & l_1 \sin(\theta_1(t)) \frac{d}{dt} \theta_2(t) \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$ 
```

```
[46]: Vhatacs # calculated earlier, above
```

```
[46]:
```


$$\begin{bmatrix} 0 & -\frac{d}{dt}\theta_1(t) - \frac{d}{dt}\theta_2(t) & 0 & l_1 \cos(\theta_1(t)) \frac{d}{dt}\theta_2(t) \\ \frac{d}{dt}\theta_1(t) + \frac{d}{dt}\theta_2(t) & 0 & 0 & l_1 \sin(\theta_1(t)) \frac{d}{dt}\theta_2(t) \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

```
[47]: shouldbezero4times4 = Vhatacs - Vhatacs_3
      shouldbezero4times4
```

```
[47]:  $\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$ 
```